Task Tracker API — Login & JWT Issuance

API TESTING REPORT

Postman-Based Testing of Login & JWT Authentication (ASP.NET Core Identity)

Prepared with Postman

Name	Nour Albzoor
Date	Aug/10/2026
Company	BinX
Project	Task Tracker API — Week 4, Day 2
Feature Tested	Login & JWT Issuance (ASP.NET Core Identity + JWT Bearer Authentication)
Endpoints	POST /api/Auth/login, GET /api/Tasks (protected)
Tool Used	Postman, jwt.io, PowerShell

1. Introduction

This report documents the functional testing of the Login and JWT (JSON Web Token) issuance flow, part of a REST API built with ASP.NET Core Identity. The testing was carried out using Postman, a widely used tool for building, sending, and validating API requests, together with jwt.io for decoding and inspecting issued tokens and PowerShell for converting token expiration timestamps.

The goal of this testing effort is to verify that the login endpoint correctly authenticates users, that a signed JWT is issued on success, that the JWT contains the expected claims and metadata, that protected endpoints correctly enforce JWT Bearer Authentication, and that expired tokens are rejected.

Before testing began, the authentication and JWT issuance logic was implemented (see the hands-on lab summary below). The endpoint's required request bodies, validation rules, and expected response formats were then reviewed, and all requests were organized and executed in a structured Postman Collection under an Authentication folder.

2. Hands-On Lab: Implement Login & JWT Issuance

Before testing could take place, the Login endpoint and JWT issuance logic were implemented in the Task Tracker API, and JWT Bearer Authentication was configured to protect the Tasks endpoints. The lab was completed in the following steps:

1	**Implement the Login endpoint** Built a POST /api/Auth/login endpoint in AuthController using UserManager<User>, SignInManager<User>, and IConfiguration. The endpoint uses UserManager.FindByEmailAsync() to look up the user by email, returning 401 Unauthorized with "Invalid email or password" if the email does not exist.
2	**Verify credentials with SignInManager** Used SignInManager.CheckPasswordSignInAsync() to verify the supplied password against the password stored by ASP.NET Core Identity. An incorrect password returns the same generic 401

	Unauthorized response as a non-existent email, so the API never reveals whether a specific email is registered.
3	**Generate a signed JWT on successful login** On valid credentials, built a JwtSecurityToken containing sub (user ID) and email claims, plus configured issuer (TaskTrackerApi) and audience (TaskTrackerClient) values. Created a symmetric SecurityKey from the configured secret and SigningCredentials using HMAC SHA-256 to digitally sign the token.
4	**Configure token expiration** Set the token's expiration using DateTime.UtcNow.AddMinutes() with the Jwt:ExpiryMinutes value from configuration, using a short ~15-minute lifetime for testing. The token was serialized with JwtSecurityTokenHandler().WriteToken() and returned in the login response body.
5	**Configure JWT Bearer Authentication in Program.cs** Registered JWT Bearer as the default scheme via AddAuthentication() / JwtBearerDefaults.AuthenticationScheme, then configured AddJwtBearer() with TokenValidationParameters (ValidateIssuer, ValidateAudience, ValidateLifetime, ValidateIssuerSigningKey all set to true) using the configured issuer, audience, and signing key. Added app.UseAuthentication() and app.UseAuthorization() to the middleware pipeline, in that order.
6	**Protect the Tasks endpoints** Added the [Authorize] attribute to TasksController so that all requests to GET /api/Tasks require a valid JWT Bearer token, and verified the resulting authentication behavior in Postman (documented in Sections 6-7 below).

3. Objective

The objective of this testing activity is to:

- Validate that the login endpoint returns the correct HTTP status codes for valid and invalid credentials.
- Confirm that a successful login issues a signed JWT containing the expected claims (sub, email, iss, aud, exp).
- Confirm that JWT Bearer Authentication correctly protects the Tasks endpoint using the [Authorize] attribute.
- Verify that a valid JWT grants access to the protected endpoint and that requests without a token are rejected.
- Confirm that an expired JWT is correctly rejected by the token validation middleware.
- Document evidence of each test case using Postman response screenshots.

4. Testing Environment

4.1 Tools Used

- Postman — for sending requests, attaching Bearer tokens, and validating responses.
- ASP.NET Core Identity — for credential verification via SignInManager.
- System.IdentityModel.Tokens.Jwt — for generating and signing the JWT.
- jwt.io — for decoding the issued JWT and inspecting its claims.
- PowerShell — for converting the exp Unix timestamp to a readable local date/time.

4.2 Endpoints Under Test

- POST /api/Auth/login — verifies credentials and issues a signed JWT.
- GET /api/Tasks — protected endpoint requiring a valid JWT Bearer token.

4.3 Base URL Configuration

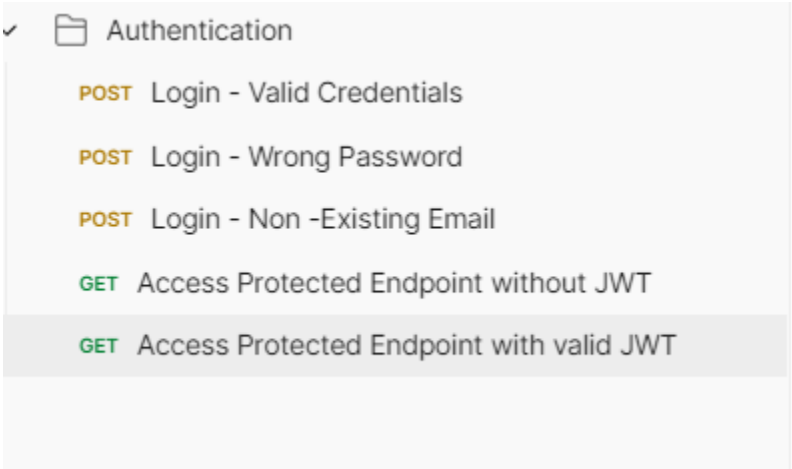
A Postman environment variable named {{baseUrl}} was used in place of a hardcoded host, so the collection can be reused across local, staging, or production environments by updating a single variable.

Example of variable usage in a request URL:

{{baseUrl}}/api/Auth/login

5. Postman Collection Structure

All requests were organized inside a single Postman Collection, under an Authentication folder :



Screenshot of the Postman collection sidebar showing the Authentication folders

6. Test Cases — Login & Protected Endpoint

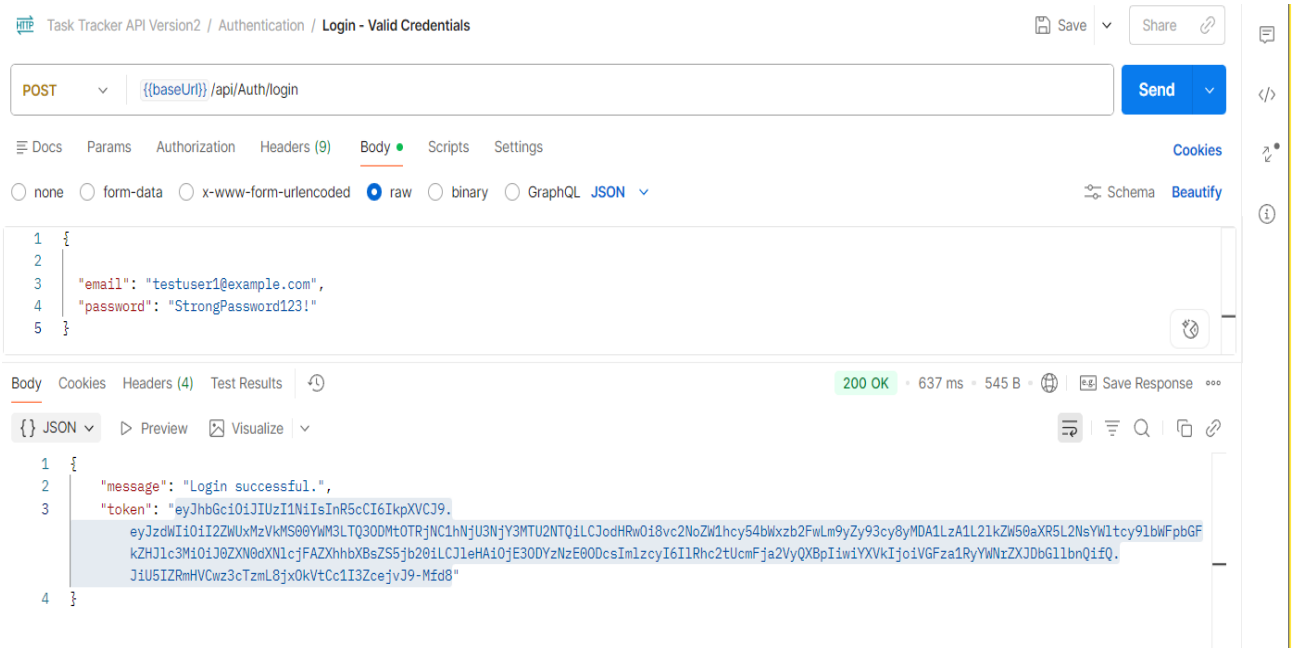
The table below summarizes all test cases executed against the login endpoint and the JWT-protected Tasks endpoint, covering successful and failed login scenarios as well as authorized, unauthorized, and expired-token access to the protected resource.

Test ID	Method	Endpoint	Expected Result
TC-001	**POST**	/api/Auth/login (valid credentials)	200 OK + JWT
TC-002	**POST**	/api/Auth/login (wrong password)	401 Unauthorized
TC-003	**POST**	/api/Auth/login (non-existing email)	401 Unauthorized
TC-004	**GET**	/api/Tasks (no Authorization header)	401 Unauthorized
TC-005	**GET**	/api/Tasks (valid Bearer JWT)	200 OK
TC-006	**GET**	/api/Tasks (expired Bearer JWT)	401 Unauthorized

6.1 TC-001 — Valid Login

Request Name	Login - Valid Credentials
Method	POST
Endpoint	/api/Auth/login
Body Type	JSON
Input	Registered email (testuser1@example.com) and correct password
Expected Result	200 OK with a signed JWT and "Login successful" message
Actual Result	200 OK — JWT issued containing sub, email, iss, aud, exp claims
Status	✓ Pass

A POST request was sent to the login endpoint with a registered user's correct email and password. UserManager.FindByEmailAsync() located the user and SignInManager.CheckPasswordSignInAsync() confirmed the password was correct. The API returned 200 OK with a signed JWT in the response body, along with the message "Login successful." The returned token contained the user ID 6ee135d1-4ac7-4783-94c4-a65766715654 and the email testuser1@example.com, which were later confirmed by decoding the token in Section 7. This token was copied and reused for the protected-endpoint tests in TC-005 and TC-006.



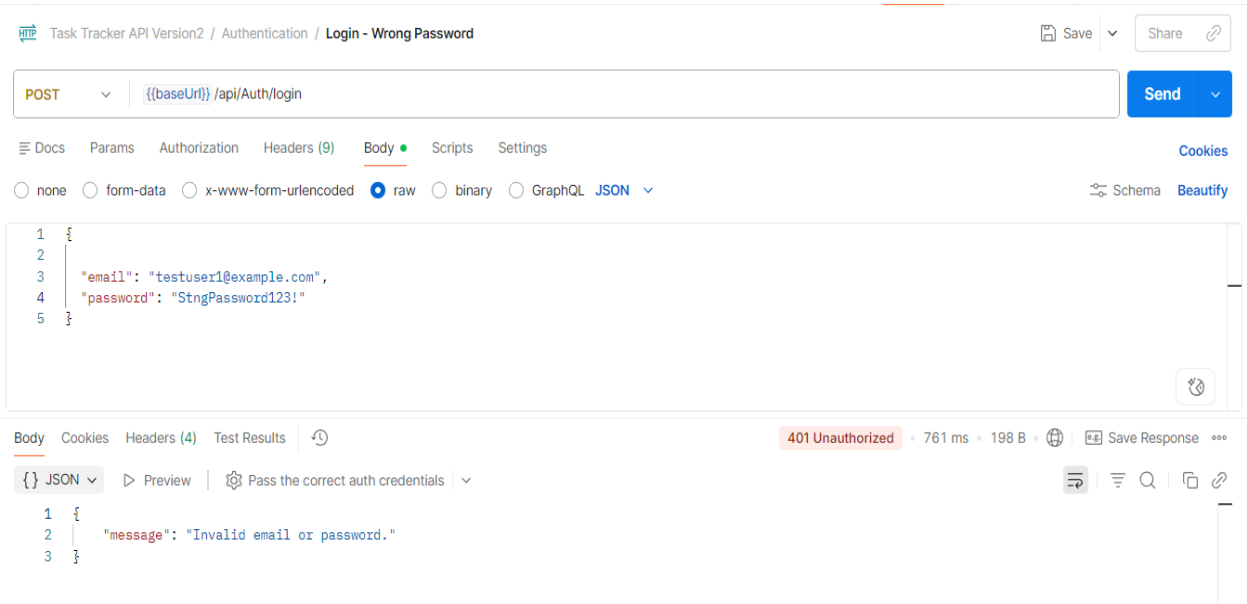
Screenshot of the Postman request/response for "Login - Valid Credentials" showing the 200 OK response and returned JWT

6.2 TC-002 — Wrong Password

Request Name	Login - Wrong Password
Method	POST

Endpoint	/api/Auth/login
Body Type	JSON
Input	Existing registered email, intentionally incorrect password
Expected Result	401 Unauthorized — "Invalid email or password"
Actual Result	401 Unauthorized — "Invalid email or password"
Status	✓ Pass

A second POST request was sent using an existing user's email but an incorrect password. SignInManager.CheckPasswordSignInAsync() returned a failed result, and the API correctly returned 401 Unauthorized with the generic "Invalid email or password" message rather than issuing a JWT.

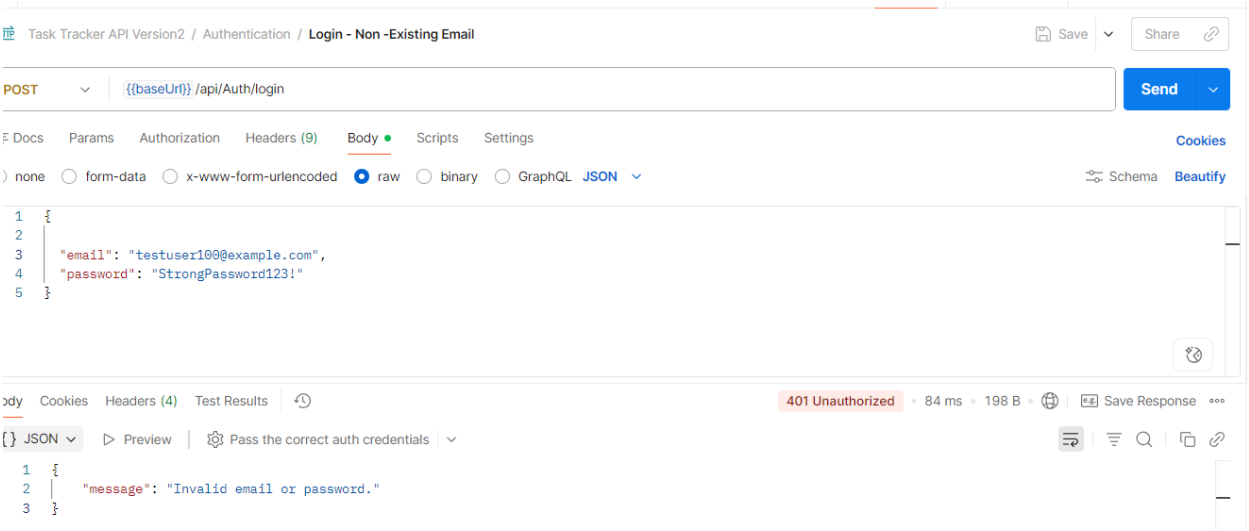


Screenshot of the Postman request/response for "Login - Wrong Password" showing the 401 Unauthorized response

6.3 TC-003 — Non-Existing Email

Request Name	Login - Non-Existing Email
Method	POST
Endpoint	/api/Auth/login
Body Type	JSON
Input	Email address not registered in the system
Expected Result	401 Unauthorized — "Invalid email or password"
Actual Result	401 Unauthorized — "Invalid email or password"
Status	✓ Pass

A third POST request was sent with an email address that does not exist in the system. UserManager.FindByEmailAsync() returned null, and the API returned the same 401 Unauthorized response and generic message used for the wrong-password case in TC-002. This confirms the API does not reveal whether a given email is registered, which prevents user enumeration.

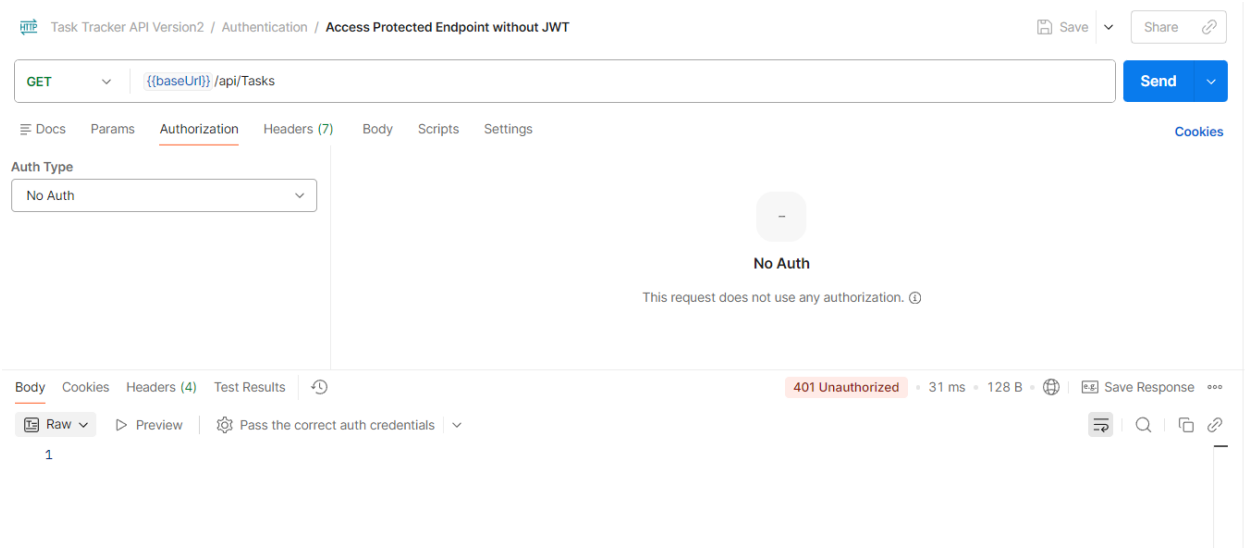


Screenshot of the Postman request/response for "Login - Non-Existing Email" showing the 401 Unauthorized response

6.4 TC-004 — Protected Endpoint Without Authentication

Request Name	Get Tasks - No Auth
Method	GET
Endpoint	/api/Tasks
Authorization	No Auth (no Bearer token attached)
Expected Result	401 Unauthorized
Actual Result	401 Unauthorized
Status	✓ Pass

A GET request was sent to the protected Tasks endpoint with Postman's Authorization set to No Auth. Because the request carried no Bearer token, JWT Bearer Authentication could not establish an identity, and the [Authorize] attribute rejected the request with 401 Unauthorized. This confirmed that the Tasks endpoint is not accessible without authentication.

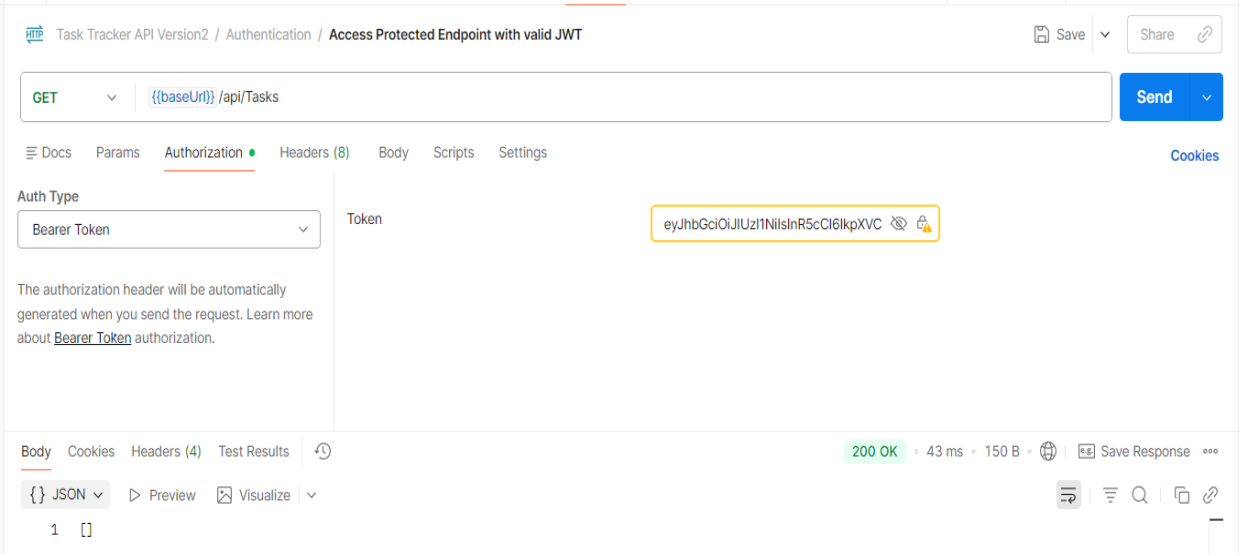


Screenshot of the Postman request/response for "Get Tasks - No Auth" showing the 401 Unauthorized response

6.5 TC-005 — Protected Endpoint With a Valid JWT

Request Name	Get Tasks - Valid JWT
Method	GET
Endpoint	/api/Tasks
Authorization	Bearer Token — JWT copied from the TC-001 login response
Expected Result	200 OK with the list of tasks
Actual Result	200 OK with the list of tasks
Status	✓ Pass

In Postman, Authorization was set to Bearer Token and the JWT obtained from the successful login (TC-001) was pasted into the Token field; Postman automatically added the "Bearer" prefix to the Authorization header. JWT Bearer Authentication validated the token's signature, issuer, audience, and expiration, and [Authorize] allowed the request through, returning 200 OK with the expected task data. This confirmed the complete authentication flow end to end.

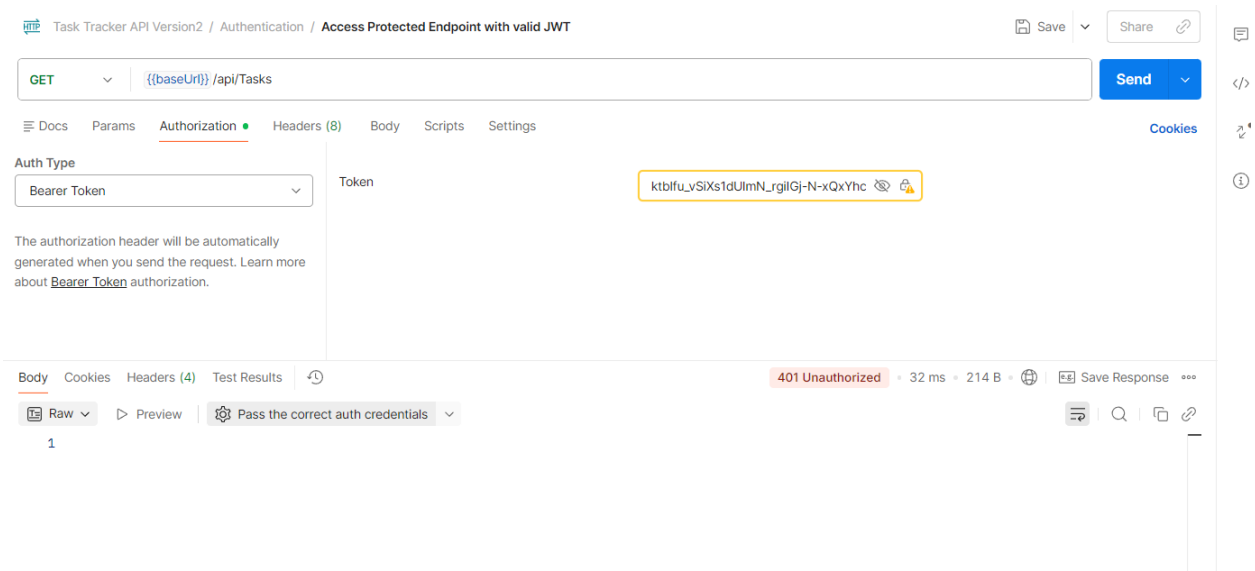


Screenshot of the Postman request/response for "Get Tasks - Valid JWT" showing the 200 OK response

6.6 TC-006 — Protected Endpoint With an Expired JWT

Request Name	Get Tasks - Expired JWT
Method	GET
Endpoint	/api/Tasks
Authorization	Bearer Token — a previously issued JWT past its exp time
Expected Result	401 Unauthorized
Actual Result	401 Unauthorized
Status	✓ Pass

An older JWT whose expiration time had already passed was attached as the Bearer token and sent to GET /api/Tasks. Because ValidateLifetime = true is configured in TokenValidationParameters, the middleware detected that the token's exp claim was in the past and rejected the request with 401 Unauthorized, even though the token's signature, issuer, and audience were otherwise valid. This confirmed that token expiration is being actively enforced.

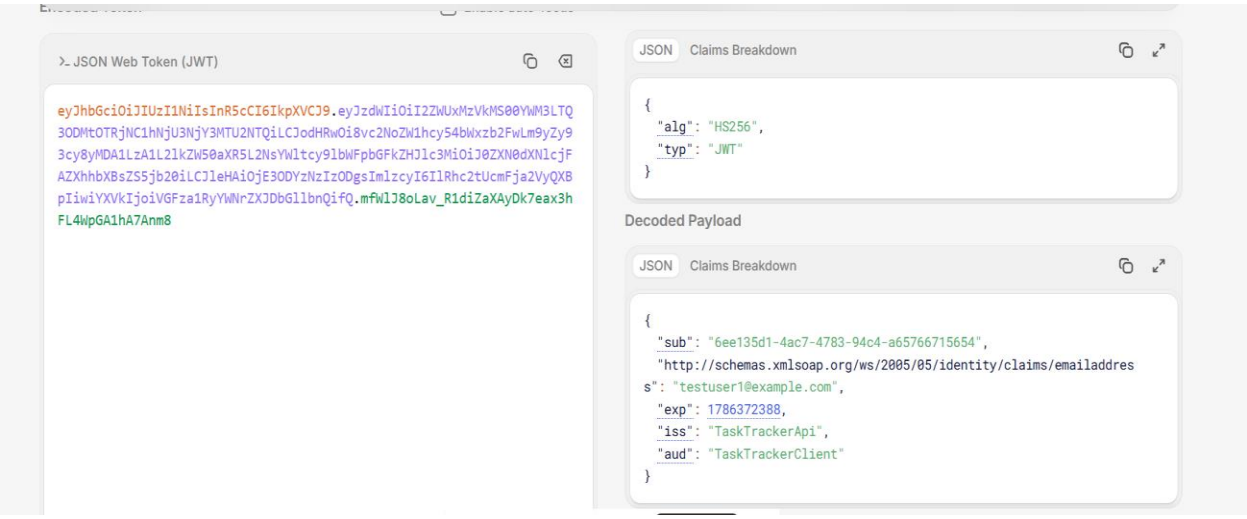


Screenshot of the Postman request/response for "Get Tasks - Expired JWT" showing the 401 Unauthorized response

7. JWT Inspection & Expiration Verification

The JWT issued in TC-001 was decoded using jwt.io to confirm it contained the claims and metadata configured in the application, rather than relying solely on the application code.

sub (user ID)	6ee135d1-4ac7-4783-94c4-a65766715654
email	testuser1@example.com
iss (issuer)	TaskTrackerApi
aud (audience)	TaskTrackerClient
exp (expiration, Unix)	1786372388



Screenshot of the decoded JWT on jwt.io showing the sub, email, iss, aud, and exp claims

8. Test Summary

A total of 6 test cases were executed across the login flow and the JWT-protected Tasks endpoint, covering both success and failure scenarios for authentication and token validation.

Total Test Cases	**6**
Login Endpoint Tests	**3**
Protected Endpoint Tests	**3**
Success Scenarios	**2**
Error Scenarios	**4**

9. Conclusion

All planned test cases were executed successfully using Postman, with results matching the expected HTTP status codes. The login endpoint correctly issues a signed JWT for valid credentials and returns a generic 401 Unauthorized for both an incorrect password and a non-existent email, without revealing which condition caused the failure.

JWT Bearer Authentication correctly protects the Tasks endpoint: requests without a token are rejected, requests with a valid token succeed, and requests with an expired token are rejected even though the signature, issuer, and audience remain valid. Decoding the token with jwt.io and converting its expiration with PowerShell confirmed that the sub, email, iss, aud, and exp claims matched the values configured in the application.

This confirms that the login and JWT issuance flow, together with the JWT Bearer Authentication middleware, meet the expected functional requirements for authenticating users and protecting API resources. Screenshot evidence for each test case is provided in the sections above and should be inserted at the indicated placeholders.